

Module 05: Handling Class Imbalance & Advanced Sampling

Lecturer: Abdan Hafidz

Topic: Strategies for Skewed Targets & Anomaly Detection

1 Introduction: The Fraud Detection Paradox

In the realm of Machine Learning, few challenges are as deceptive as the Class Imbalance problem. To illustrate this, imagine you are a Data Scientist at a major bank tasked with building an Artificial Intelligence system to detect Credit Card Fraud. You are given a massive dataset containing one million distinct transactions. Upon exploring the data, you discover a stark reality: 990,000 of these transactions are legitimate, everyday purchases, while only 10,000 are fraudulent activities.

This creates a dangerous statistical trap known as the **Accuracy Paradox**. If you were to train a naive, "dumb" model that simply learned to predict "Legit" for every single transaction it saw, it would achieve a staggering **99% Accuracy**. Mathematically, this is correct because it correctly identified the 99% of normal data. However, practically, this model is catastrophic. It failed to detect a single fraudster, allowing millions of dollars to be stolen while the data science team celebrates their high accuracy score. This phenomenon highlights a critical lesson: when the target variable is heavily skewed, accuracy becomes a meaningless, vanity metric. We must dig deeper to find the truth.

1.1 Mathematical Abstraction: The Imbalance Ratio

Let D be a dataset with N samples, where $y_i \in \{0, 1\}$ represents the class label for the i -th sample.

- N_{maj} : Number of samples in the Majority Class (Legit, $y = 0$).
- N_{min} : Number of samples in the Minority Class (Fraud, $y = 1$).

The **Imbalance Ratio (IR)** is defined as:

$$IR = \frac{N_{maj}}{N_{min}}$$

In our bank example, $IR = \frac{990,000}{10,000} = 99$. An $IR \gg 1$ indicates severe imbalance, necessitating the specialized techniques discussed in this module.

2 The Confusion Matrix: Deconstructing Prediction Errors

To properly evaluate a model in an imbalanced environment, we must abandon simple accuracy and adopt a more granular framework known as the **Confusion Matrix**. This matrix decomposes the model's performance into four distinct quadrants, allowing us to understand not just *if* the model made an error, but *what kind* of error it made.

The first quadrant is the **True Positive (TP)**. In our fraud context, this represents the ideal scenario: a fraudulent transaction occurred, and the model correctly sounded the alarm. The opposite of this is the **True Negative (TN)**, where a legitimate transaction occurred, and the model correctly let it pass. These are the successful outcomes.

However, the errors tell a more important story. A **False Positive (FP)**, often called a "False Alarm," occurs when the model flags a legitimate transaction as fraud. While this is inconvenient for the customer—who might receive a text message asking if they really bought coffee in Paris—it is generally a safe failure. The bank loses no money, only a small amount of customer patience.

The most dangerous quadrant is the **False Negative (FN)**. This is a "Missed Detection." Here, a fraudster strikes, but the model predicts "Legit." The transaction goes through, the money is lost, and the bank incurs a liability. In fraud detection, our primary engineering goal is to minimize these False Negatives, even if it means accepting a few more False Alarms (False Positives).

		Actual: Fraud (Positive)	Actual: Legit (Negative)
Pred:	Fraud	True Positive (TP) (Hit) <i>Fraud correctly detected</i>	False Negative (FN) (Miss - DANGER!) <i>Fraud labeled as Legit</i>
	Legit	False Positive (FP) (False Alarm) <i>Legit labeled as Fraud</i>	True Negative (TN) (Correct Rejection) <i>Legit labeled as Legit</i>

Figure 1: The Confusion Matrix. In domains like Fraud Detection or Cancer Diagnosis, the cost of a False Negative is significantly higher than a False Positive.

2.1 Formalizing Metrics: Precision, Recall, and F1

From the Confusion Matrix, we derive metrics that are robust to imbalance.

1. **Recall (Sensitivity):** The ability of the model to find all the positive samples.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Interpretation: "Out of all the actual frauds, how many did we catch?" A high Recall means fewer missed frauds (low FN).

2. **Precision:** The ability of the model not to label a negative sample as positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Interpretation: "Out of all the transactions we flagged as fraud, how many were actually fraud?" A high Precision means fewer false alarms (low FP).

3. **F1-Score:** The harmonic mean of Precision and Recall. It punishes extreme values.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

This is the single best metric for imbalanced datasets when you need a balance between catching fraud and avoiding false alarms.

3 Sampling Techniques: Rebalancing the Scales

Since standard algorithms (like Logistic Regression or Random Forests) tend to be biased toward the majority class, we must intervene by modifying the training data itself. We have three primary techniques to artificially balance the scales of justice within our dataset.

3.1 Random Undersampling

The most straightforward approach is **Random Undersampling**. Here, we indiscriminately remove examples from the majority class (Legit transactions) until the number of legit transactions matches the number of fraud transactions. If we have 10,000 frauds, we keep only 10,000 legit transactions and discard the other 990,000.

While this makes the dataset perfectly balanced and drastically reduces training time, it comes with a severe penalty: **Information Loss**. By throwing away 99% of our legitimate data, we are discarding vast amounts of valuable patterns. The model becomes "dumber" regarding normal behavior because it has seen so few examples of it. This technique is generally recommended only when you have an overwhelming amount of data (e.g., hundreds of millions of rows) where dropping some doesn't hurt the overall distribution.

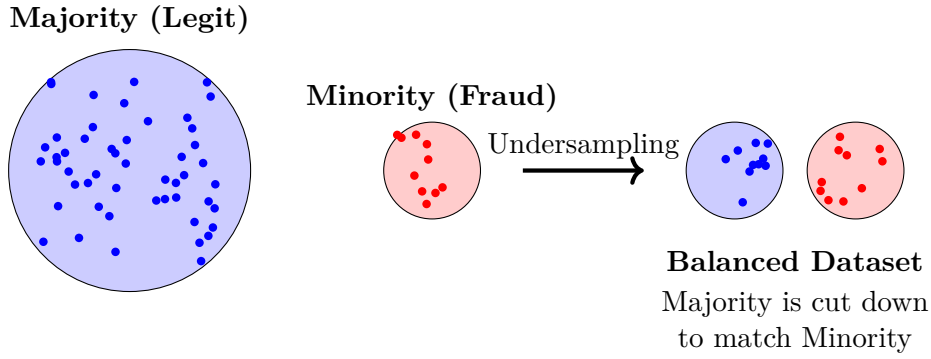


Figure 2: Random Undersampling: Reducing the Majority Class size causes significant data loss but ensures balance.

3.2 Random Oversampling

The inverse approach is **Random Oversampling**. Instead of deleting the majority, we duplicate the minority. We take our 10,000 fraud cases and copy-paste them repeatedly until we have 990,000 fraud cases to match the legit ones.

The advantage here is that we lose no information; every single original data point is preserved. However, this technique introduces the risk of **Overfitting**. Because the model sees the exact same fraud examples over and over again, it memorizes them rather than learning the general characteristics of fraud. It becomes excellent at detecting the specific frauds it has seen before but fails to generalize to new, unseen fraud patterns.

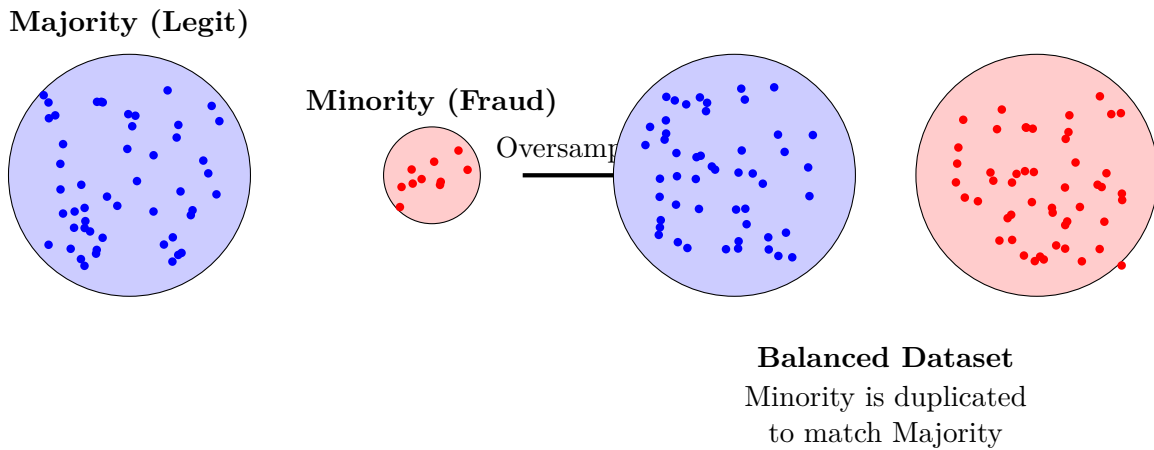


Figure 3: Random Oversampling: Duplicating the Minority Class creates balance without data loss, but risks overfitting due to repeated data points.

3.3 SMOTE (Synthetic Minority Over-sampling Technique)

To solve the overfitting problem of simple oversampling, we use a more sophisticated mathematical approach called **SMOTE**. Instead of mere duplication, SMOTE uses linear algebra to hallucinate new, plausible examples of fraud.

The algorithm works by selecting a fraud data point and identifying its "Nearest Neighbors" in

the feature space (using the K-Nearest Neighbors algorithm). It then draws a vector (a line) between the existing fraud point and its neighbor. Finally, it picks a random point along this line to create a new, synthetic data point. This effectively "thickens" the minority cluster, filling in the empty space between existing examples with mathematically consistent synthetic data. This helps the model generalize better because it is training on new variations of fraud rather than exact copies.

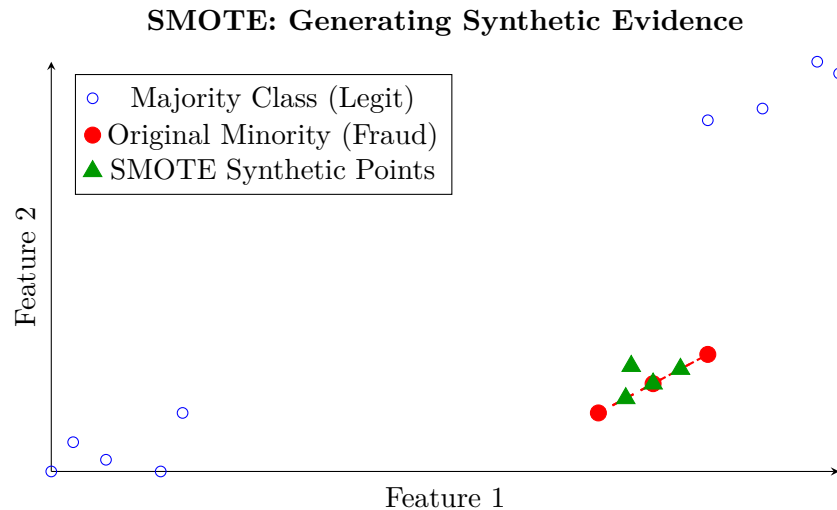


Figure 4: Visualizing SMOTE. The green triangles are not duplicates; they are mathematically generated "hybrids" created by interpolating between the red original points.

4 The Future: Handling Unlabeled Data

The techniques discussed so far assume that we know which transactions are fraud and which are legit. But what if we encounter a dataset that has no labels at all? Imagine receiving a raw dump of server logs or customer behaviors without any "Target Column."

This leads us to the domain of **Unsupervised Learning**, which we will explore in the next chapter. Even without labels, we can use algorithms to find hidden structures in the data. Techniques like **Clustering** (K-Means, DBSCAN) can analyze the geometry of the data and automatically group similar points together. They might discover that 99% of your data points form a tight, dense cluster (Normal Behavior), while 1% are scattered far away in the periphery. These isolated points are mathematical anomalies, and in many real-world scenarios, these anomalies turn out to be exactly what we are looking for—fraud, defects, or cyberattacks.

Once these clusters are identified, we can apply **Stratified Sampling**. This advanced sampling technique ensures that when we split our data into training and testing sets, we preserve the exact ratio of these clusters. If the original world contains 1% anomalies, Stratified Sampling guarantees that both our training set and our test set will also contain exactly 1% anomalies, preventing the statistical bias that can ruin a model's validity.

5 Implementation Guide

Below is a robust Python implementation using the ‘imbalanced-learn’ library. Note the strict separation of training and testing data to prevent data leakage.

```
1 from imblearn.over_sampling import RandomOverSampler, SMOTE
2 from imblearn.under_sampling import RandomUnderSampler
3 from sklearn.model_selection import train_test_split
4
5 # 1. THE GOLDEN RULE: Split FIRST
6 # We must split the data before resampling to ensure the Test Set
7 # remains pure and representative of the real world.
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y, test_size=0.2, stratify=y, random_state=42
10 )
11
12 # 2. Apply Undersampling (High Information Loss)
13 # Useful for massive datasets (millions of rows)
14 rus = RandomUnderSampler(random_state=42)
15 X_resampled_rus, y_resampled_rus = rus.fit_resample(X_train, y_train)
16
17 # 3. Apply Oversampling (High Overfitting Risk)
18 # Useful as a baseline, but rarely the best final choice
19 ros = RandomOverSampler(random_state=42)
20 X_resampled_ros, y_resampled_ros = ros.fit_resample(X_train, y_train)
21
22 # 4. Apply SMOTE (The Balanced Choice)
23 # Generates synthetic data to bridge the gap
24 smote = SMOTE(random_state=42)
25 X_resampled_smote, y_resampled_smote = smote.fit_resample(X_train, y_train)
26
27 # Now, we train the model on X_resampled_smote,
28 # BUT we evaluate it on the original X_test.
```

CRITICAL WARNING: DATA LEAKAGE

You must **NEVER** apply SMOTE or Oversampling to your Test Set. The Test Set serves as a simulation of the “Real World.” In the real world, fraud is rare (1%). If you artificially balance your Test Set to be 50% fraud, your evaluation metrics will be completely fake and overly optimistic. **Always Split Data → Resample Training Data → Train Model → Test on Original Test Data.**